

大模型基础

一、大模型评估

BLEU、ROUGE都能够从某一侧面评估大模型的质量，但是都不全面。BLEU、ROUGE受表达字词及排列顺序的影响较大，对字词不一致但语义高度相似的文本评估时可能会产生重大偏差。

如果多次评估都动用人力甚至领域专家，评估成本太高。

一个通用方法是使用大模型进行初步评估，将确定性结果排出后，不确定的部分由人工检查。

大模型评估的方法参加下面的提示词

你是一位语言分析专家，分析下面的【例句一】和【例句二】是否相似，并给出相似性分数。相似性分数从0到9，相似性越高，分数越接近9，相似性越低，分数越接近0。只允许输出相似性分数，不允许输出其他任何文本。

【例句一】

It is a nice day today

【例句二】

The weather is lovely this afternoon

你是一位语言分析专家，分析下面的【例句一】和【例句二】是否相似，并给出相似性分数。相似性分数从0到9，相似性越高，分数越接近9，相似性越低，分数越接近0。只允许输出相似性分数，不允许输出其他任何文本。

【例句一】

It is a nice day today

【例句二】

It is national day today

二、transformer结构要点

1 自注意力

$$\begin{array}{c} \mathbf{X} \end{array}
 \begin{array}{c} \mathbf{W}^{\mathbf{Q}} \end{array}
 \begin{array}{c} \mathbf{Q} \end{array}$$

$$\begin{array}{c} \mathbf{X} \end{array}
 \begin{array}{c} \mathbf{W}^{\mathbf{K}} \end{array}
 \begin{array}{c} \mathbf{K} \end{array}$$

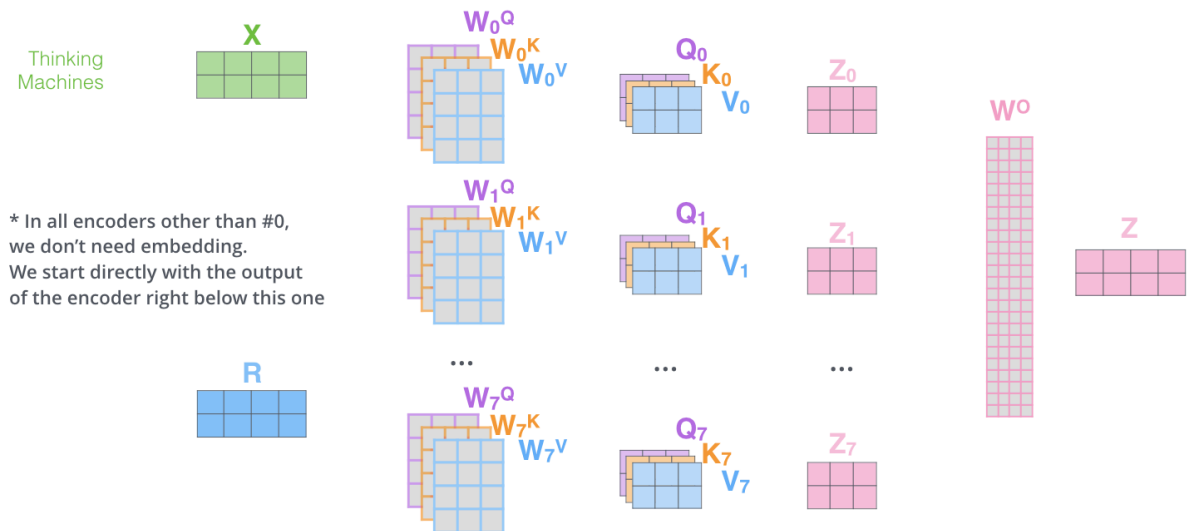
$$\begin{array}{c} \mathbf{X} \end{array}
 \begin{array}{c} \mathbf{W}^{\mathbf{V}} \end{array}
 \begin{array}{c} \mathbf{V} \end{array}$$

$$\text{softmax} \left(\frac{\begin{array}{c} \mathbf{Q} \end{array} \times \begin{array}{c} \mathbf{K}^{\mathbf{T}} \end{array}}{\sqrt{d_k}} \right) \begin{array}{c} \mathbf{V} \end{array}$$

$$= \begin{array}{c} \mathbf{Z} \end{array}$$

2 多头注意力

- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



3 LN层

LayerNorm是针对单个样本的所有特征维度进行归一化。

对于一个输入向量 $x=(x_1, x_2, \dots, x_H)$ (H 是隐藏层的维度)，LayerNorm的计算步骤如下：

1. 计算均值和方差：计算这个样本所有维度上的均值和方差。

$$\mu = \frac{1}{H} \sum_{i=1}^H x_i$$
$$\sigma^2 = \frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2$$

2. 归一化：用计算出的均值和方差对输入进行归一化，使其均值为0，方差为1。

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

(其中 ϵ 是一个极小的数，如 $1e-5$ ，防止分母为零)

缩放和偏移：引入两个可学习的参数（缩放因子 γ 和偏移项 β ），对归一化后的值进行线性变换。

$$y_i = \gamma \hat{x}_i + \beta$$

这一步至关重要，因为它赋予了模型恢复表征能力的自由度。如果规范化操作丢失了某些重要信息（比如激活函数的非线性区域），模型可以通过学习 γ 和 β 来恢复它。

在Transformer架构中，LayerNorm被用在两个关键位置：

1. 每个编码器/解码器子层的后面（即残差连接之后），例如：`SublayerOutput = LayerNorm(x + Sublayer(x))`。
2. 在最终输出层之前。

其优势体现在：

- 对Batch Size不敏感：Transformer模型常常需要处理变长序列，Batch内的序列长度可能差异很大。BatchNorm在这种场景下效果不佳，而LayerNorm在每个样本上独立计算，与Batch Size无关，更加稳定。
- 适用于变长序*：无论是长句子还是短句子，LayerNorm都在其自身的特征维度上操作，处理起来非常自然。
- 稳定训练：Transformer模型通常很深，没有LayerNorm，训练非常困难。LayerNorm确保了数据流经多层后分布依然相对稳定。
- 与残差连接协同工作：LayerNorm被放在残差连接之后，可以规范化“原始输入”和“层变换输出”的叠加结果，使得数据在深度网络中能够更顺畅地流动。

三、BERT的改进

1 预训练任务

1.1 完形填空

输入序列：

首先，将一段文本（比如一个或两个句子）输入到BERT中。

例如：`"今天天气很好，我们一起去公园玩。"`

随机掩码：

随机选择输入序列中15%的单词（Token）进行遮盖。遮盖有三种方式，按一定概率随机选择：

- 80%的概率
：将单词替换为特殊的

`[MASK]`

标记。

- 例如：`"今天天气很[MASK]，我们一起去公园玩。"`
- 10%的概率
：将单词替换为一个随机的其他单词。
 - 例如：`"今天天气很**棒**，我们一起去公园玩。"`（假设随机把“好”换成了“棒”）
- 10%的概率
：保持单词不变。
 - 例如：`"今天天气很好，我们一起去公园玩。"`（“好”没有被改变）

优势

- 双向上下文理解：这是MLM最革命性的地方。传统的语言模型（如GPT）是单向的（从左到右或从右到左），只能看到目标词之前的信息。而BERT的MLM任务允许模型同时看到目标词左右两边的全部信息，从而获得更深刻、更全面的语言理解能力。

- 深层语义学习：为了准确预测被遮盖的词，模型必须学习单词之间的语法关系、语义关联、常见搭配等语言学知识。例如，要预测“我早上喝了一杯[MASK]”，模型需要知道“咖啡”、“牛奶”、“水”等词比“汽车”、“石头”更合理。
- 为下游任务打下基础：MLM任务本质上是一个“理解”任务，这与很多下游任务（如文本分类、情感分析、问答系统）的要求高度一致，即需要模型理解整个句子的含义。因此，通过MLM预训练出的BERT模型，其编码器已经具备了强大的语言表征能力，只需稍作微调就能在各种任务上取得优异效果。

1.2 句对任务

下一句预测（Next Sentence Prediction, NSP）任务的目标是让模型理解两个句子之间的关系，特别是判断第二个句子是否是第一个句子的自然延续。这旨在训练模型捕捉句子级别的语义连贯性，而不仅仅是词汇级别的信息。

在预训练时，BERT的输入是由两个句子A和B拼接而成的。

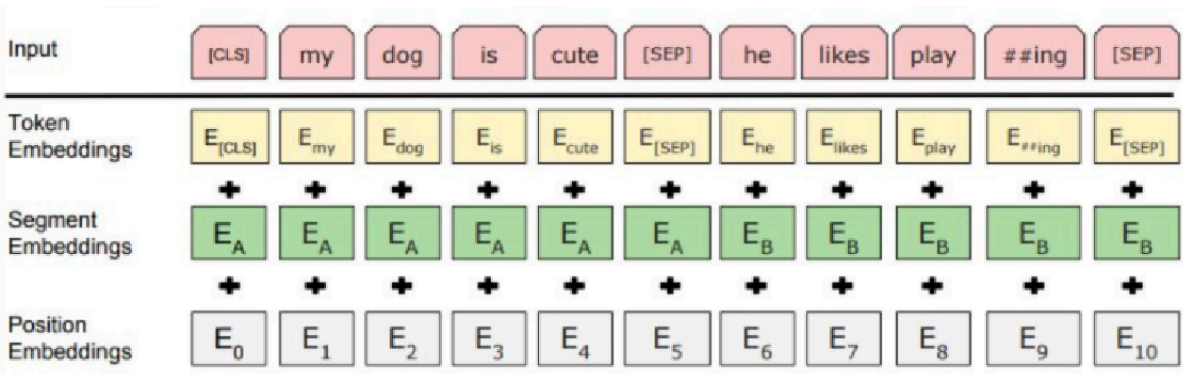
- 正例 (IsNext)
 - ：从语料库（如维基百科）中选取连续的两个句子。句子B确实是句子A的下一句。
 - 例如：
 - 句子A: “月球是地球唯一的天然卫星。”
 - 句子B: “它围绕着地球进行公转。”
- 负例 (NotNext)
 - ：从语料库中随机抽取一个句子作为句子B，它和句子A在原文中不连续
 - 例如：
 - 句子A: “月球是地球唯一的天然卫星。”
 - 句子B: “咖啡是一种用烘焙过的咖啡豆制成的饮料。”（一个随机抽取的无关句子）

在训练数据中，正例和负例各占50%。

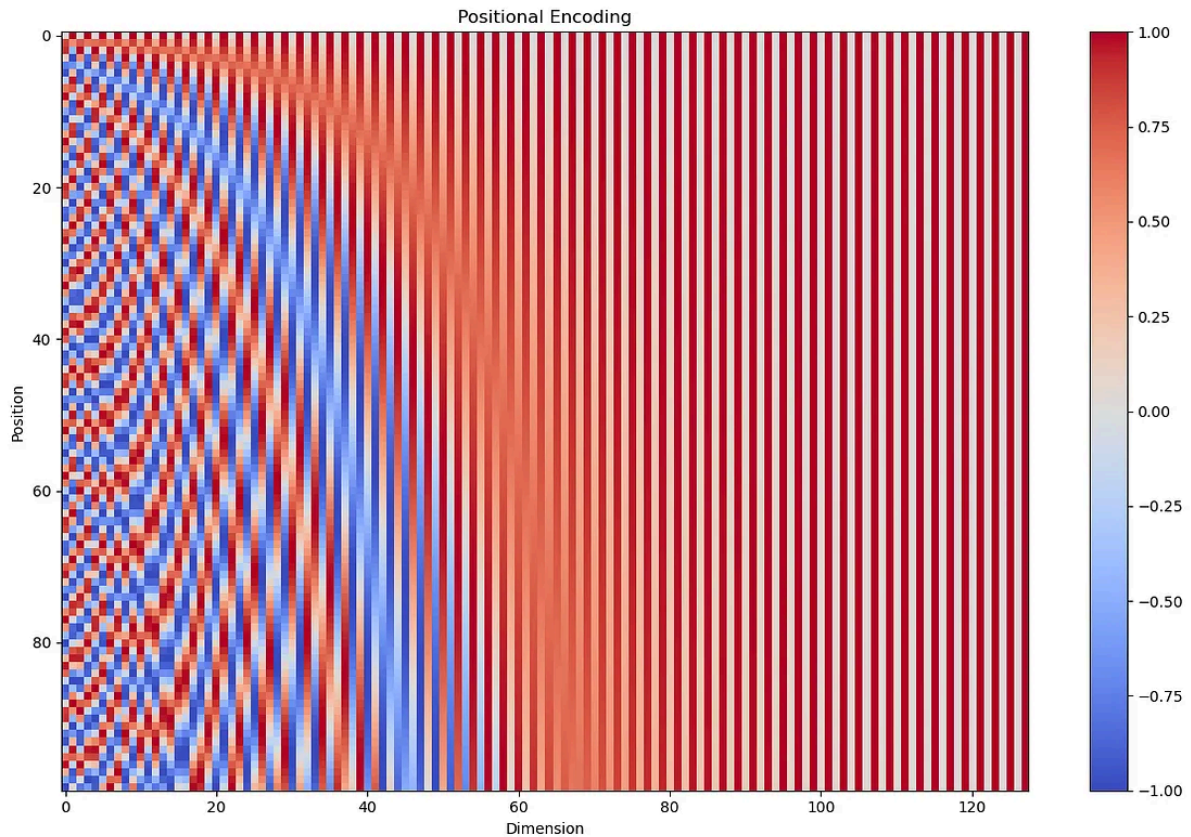
BERT使用特定的标记来构建输入：

- [CLS] 标记放在整个序列的开头。
- 句子A和句子B之间用 [SEP] 标记分隔。
- 最终输入形式：[CLS] 句子A [SEP] 句子B [SEP]

2 embedding层



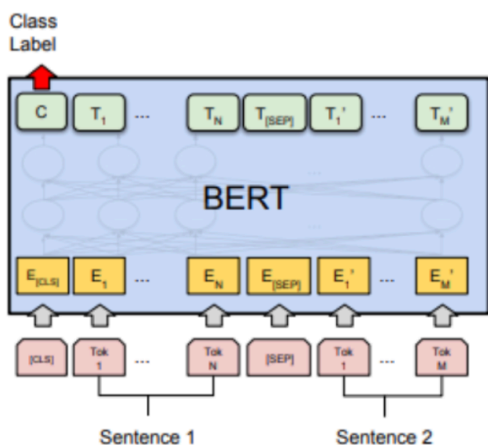
$$PE_{(pos,i)} = \begin{cases} \sin\left(\frac{pos}{10000^{i/d_{model}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{pos}{10000^{(i-1)/d_{model}}}\right) & \text{if } i \text{ is odd} \end{cases}$$



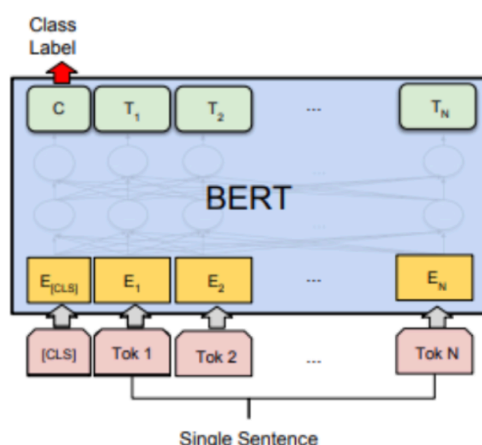
3 下游任务微调

微调是BERT模型从“通用语言理解模型”转变为“特定任务专家”的关键步骤。其核心思想是：在预训练好的BERT模型基础上，针对特定的下游任务，用有标签的任务数据对模型的所有参数进行进一步的训练。

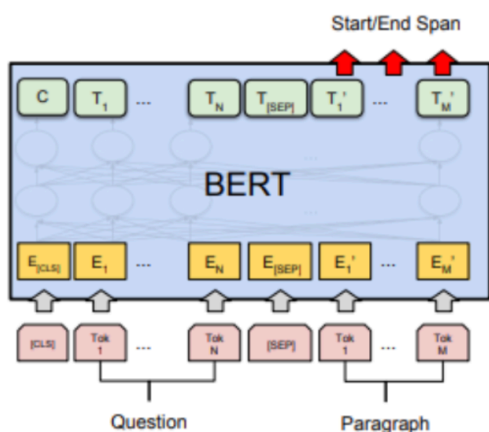
BERT的灵活性体现在它可以被微调以适应各种类型的NLP任务。



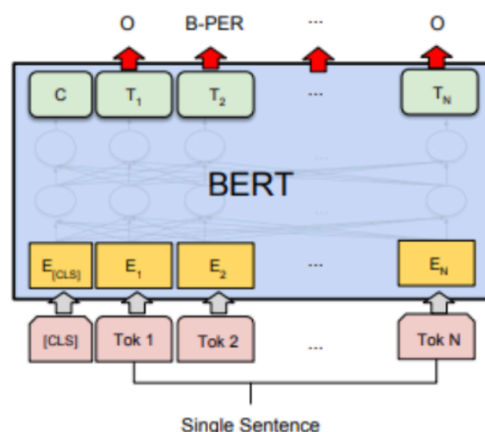
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



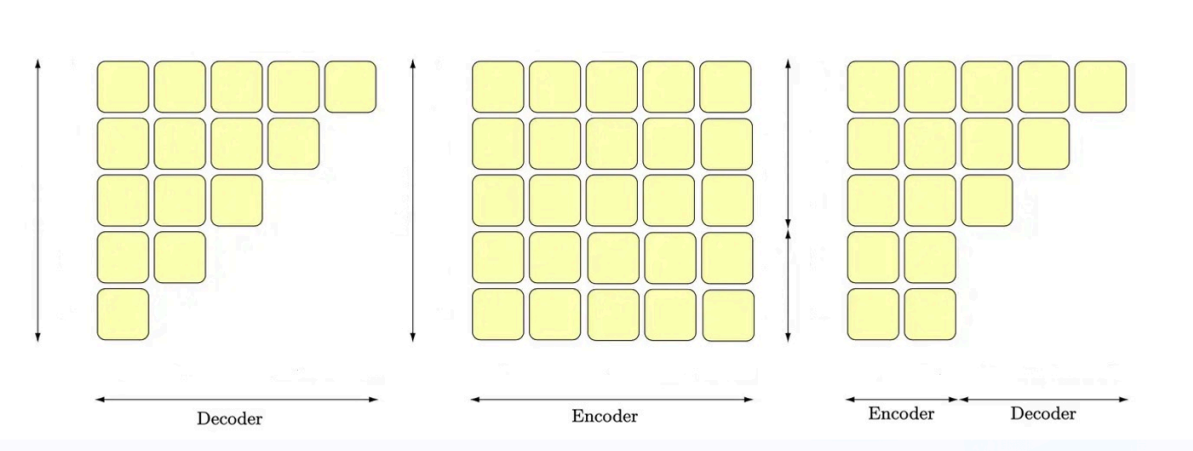
(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

任务类型	示例	输入处理与输出层
句子对分类	自然语言推理 (NLI)、语义相似度	输入：句子A [SEP] 句子B。使用 [CLS] 标记的输出向量进行分类（蕴含/中立/矛盾）或二分类（相似/不相似）。
单句分类	情感分析、垃圾邮件检测	输入：单个句子。使用 [CLS] 标记的输出向量进行情感分类（正面/负面）等。
问答	SQuAD	输入：问题 [SEP] 包含答案的文本段落。输出层需要两个向量，分别预测答案在段落中的开始位置和结束位置。
序列标注	命名实体识别、词性标注	输入：单个句子。将句子中每个词元对应的最终输出向量都送入一个分类器，预测其标签（如人名、地名等）。

四、主流大模型的发展差异



自编码模型是在知道模型上下文的情况下推测某个token的语义

自回归模型是在只知道上文的情况下推测下一个token的语义

序列到序列模型是在知道序列1的全文和序列2上文的情况下推测序列2下一个token的语义

这相当于求解下面的三个线性方程组：

$\begin{aligned} 3a+7b+4c+9d+2e &= 26 \\ 2a+5b+3c+2d &= 18 \\ 7a+6b+3c &= 42 \\ 4a+3b &= 15 \\ 8a &= 16 \end{aligned}$	$\begin{aligned} 3a+6b+9c+12d+15e &= 51 \\ 8a+2b+5c+2d+2e &= 66 \\ 2a+4b+6c+8d+10e &= 34 \\ 5a+2b+7c+d+3e &= 26 \\ a+2b+3c+4d+5e &= 17 \end{aligned}$	$\begin{aligned} 3a+7b+4c+9d+2e &= 26 \\ 2a+5b+3c+2d &= 18 \\ 7a+6b+3c &= 42 \\ 4a+3b &= 15 \\ 8a+6d &= 30 \end{aligned}$
--	---	---

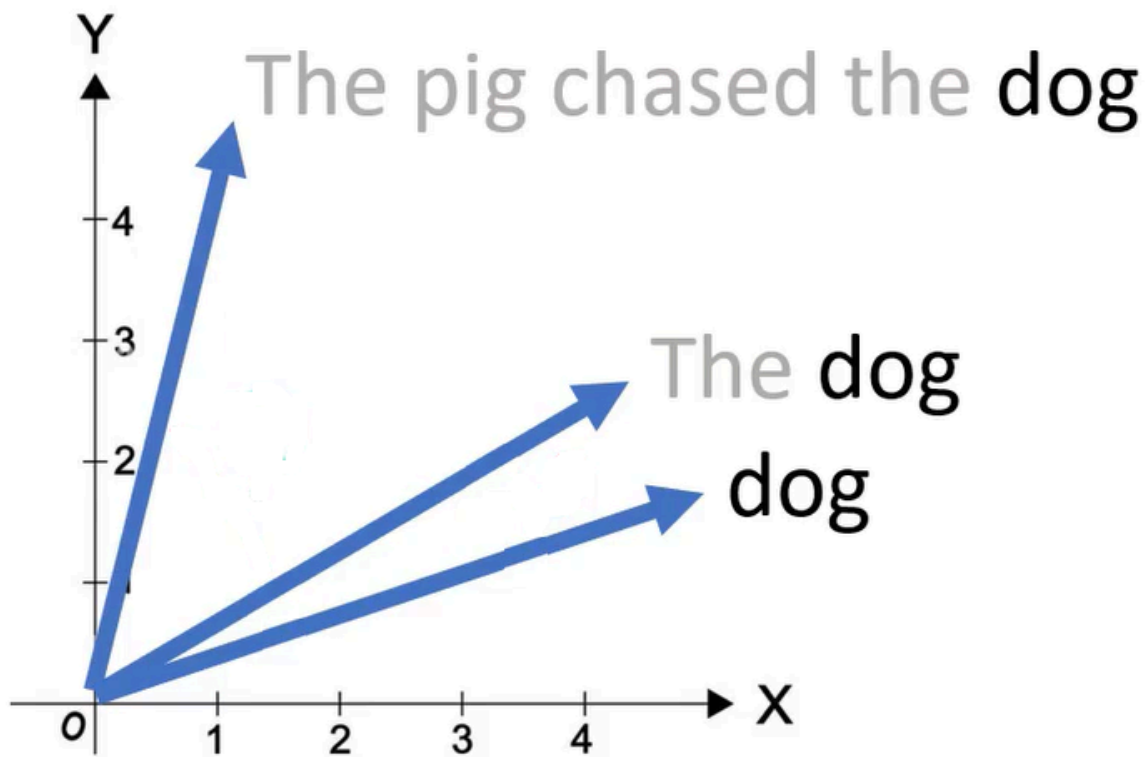
当模型比较小时，自编码模型因为能同时知道上下文信息，训练更为成熟，所以率先发展起来。

随着模型越来越大，自编码模型方程组中的共线性问题会越来越严重，表意能力受到较大的削弱，自回归模型则基本不受影响。

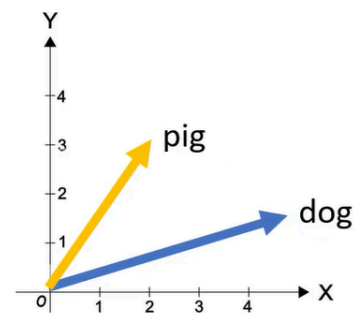
五、大模型发展过程中的重大改进

1 旋转位置编码

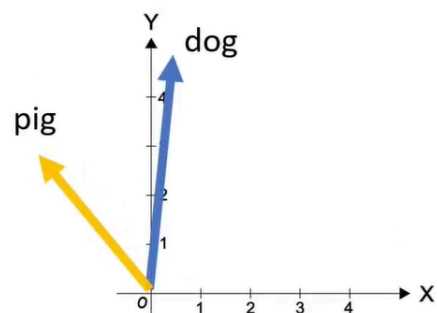
- 能较好地表达相对位置
- 计算成本低



The pig chased the dog



Once upon a time, the pig
chased the dog



这种旋转的数学表达形式为：

$$\begin{aligned}
 f_q(\mathbf{x}_m, m) &= \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_q^{(1,1)} & W_q^{(1,2)} \\ W_q^{(2,1)} & W_q^{(2,2)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix} \\
 &= \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} q_m^{(1)} \\ q_m^{(2)} \end{pmatrix}
 \end{aligned} \tag{9}$$

最终实际的计算量很小

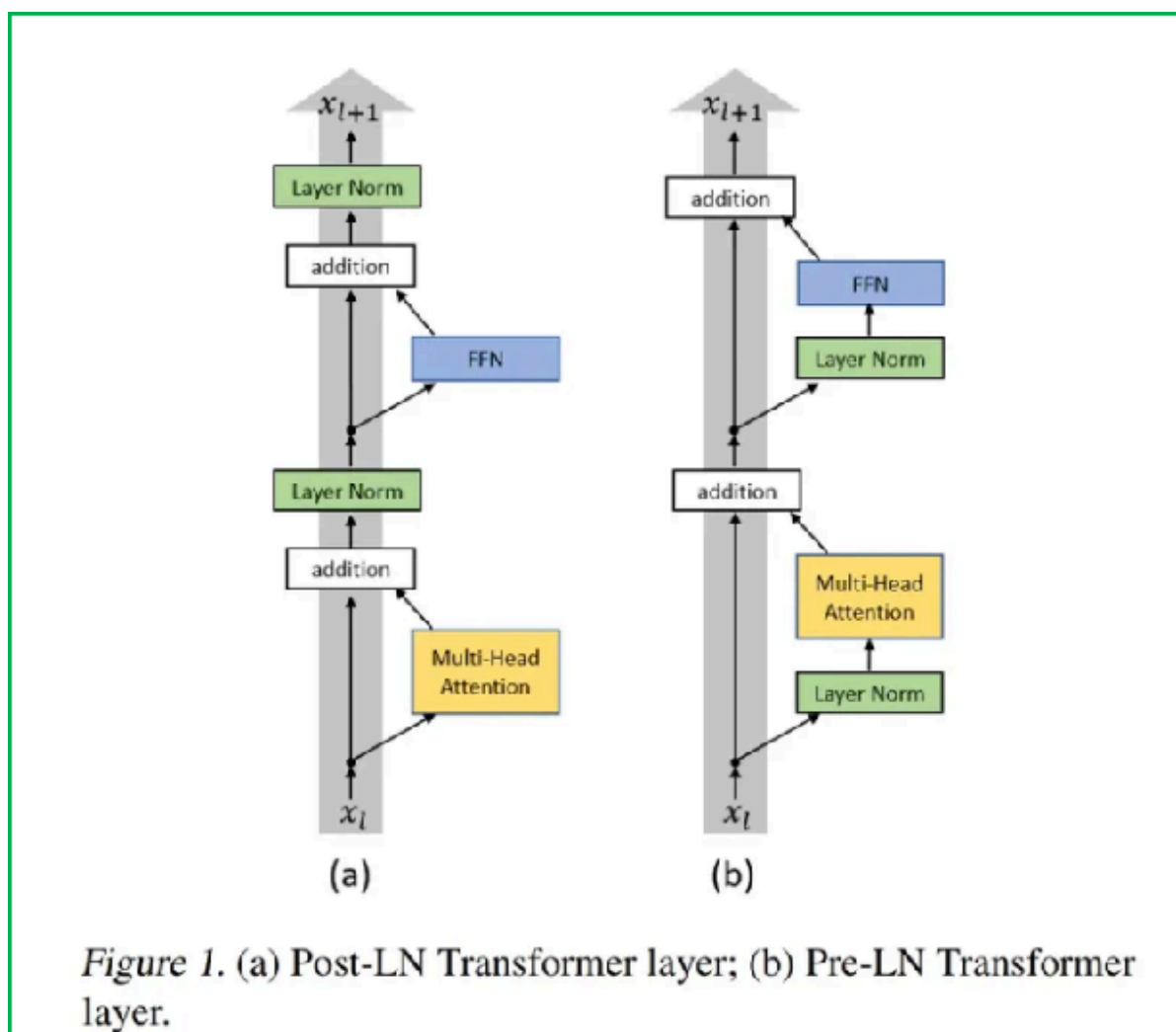
$$f_q(\mathbf{x}_m, m) = (\mathbf{W}_q \mathbf{x}_m) e^{im\theta}$$

推导详情参照这篇文章

https://www.zhihu.com/tardis/bd/art/647109286?source_id=1001

2 层数增长

2.1 pre-norm



2.2 deep-norm

$$x_{l+1} = LN(\alpha x_l + G_l(x_l, \theta_l)),$$

三者差异: post-norm(20层左右), pre-norm(100+层), deep-norm(1000层)

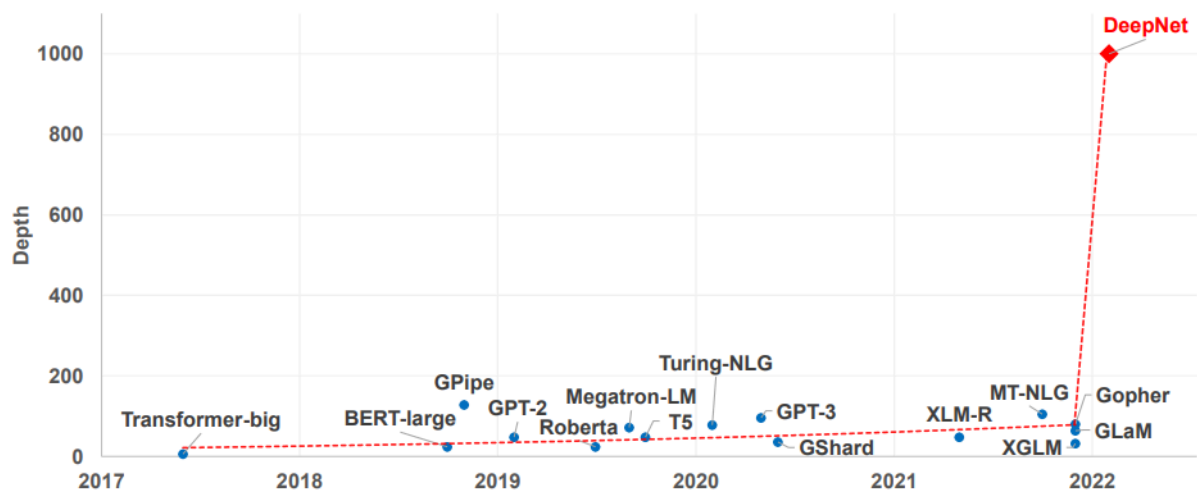
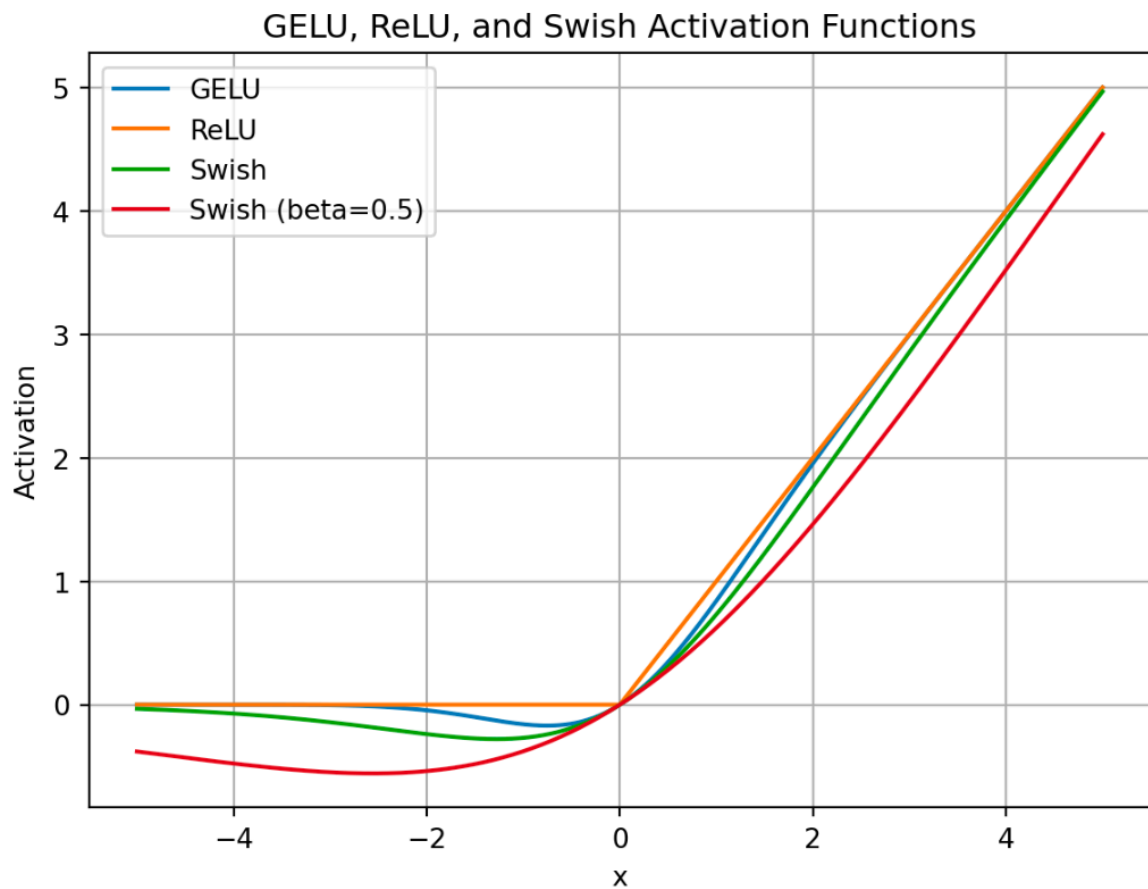


Figure 1: Trend of Transformer depths of state-of-the-art NLP models over time.

3 激活函数



$$\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$

$$\text{GEGLU}(x, W, V, b, c) = \text{GELU}(xW + b) \otimes (xV + c)$$

$$\text{Swish}_\beta(x) = x\sigma(\beta x)$$

$$\text{SwiGLU}(x, W, V, b, c) = \text{Swish}_1(xW + b) \otimes (xV + c)$$

4 RMSNorm

一般的LN层

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

(其中 ϵ 是一个极小的数, 如1e-5, 防止分母为零)

RMSNorm

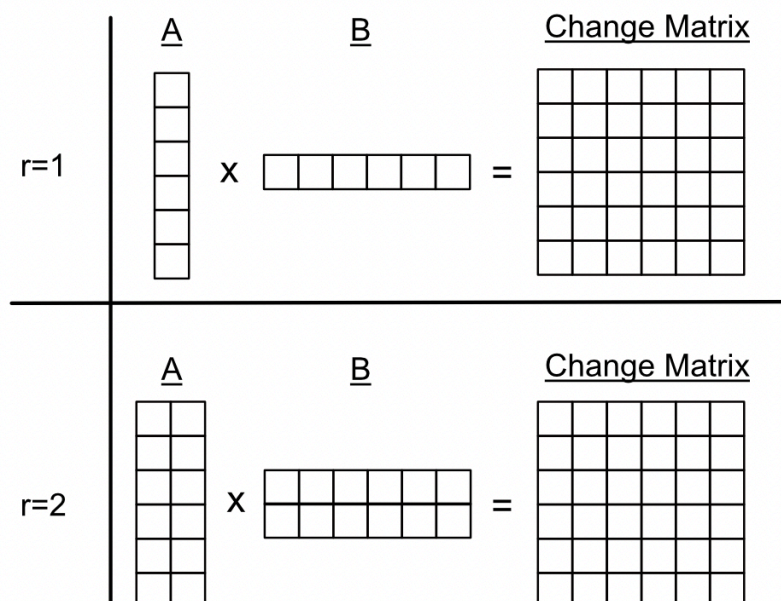
$$RMS(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}$$

$$x = \frac{x}{RMS(x)} \cdot \gamma$$

RMS Norm的主要区别在于去掉了减去均值的部分

5 多头潜在注意力

MLA用低秩因子分解的方式, 大幅降低了计算量, 从MXN 降低到MXr+rXN



6 混合专家模型

6.1 核心思路

- 传统模型（稠密模型）：相当于找一个无所不知的“通才”。你需要教会这个人关于历史、地理、语言、美食、预算等所有方面的知识。让他处理每个问题，虽然全面，但可能不够精深，而且培养这样一个“通才”成本极高（需要巨大模型参数）。
- 混合专家模型：相当于组建一个“专家委员会”。这个委员会里有：
 - 地理专家：精通各国地理位置和气候。
 - 美食专家：深谙世界各地美食。
 - 预算专家：擅长精打细算。
 - 语言专家：通晓多国语言。
 -

6.2 核心组件

一个标准的MoE层包含两个关键部分：

1. 专家：这些是小型神经网络，通常是前馈网络。一个MoE层中包含多个这样的专家（例如8个、64个，甚至更多）。每个专家都有其擅长的特定领域或数据类型。
2. 门控网络：也称为路由器。它的作用是分析输入数据，并决定将数据发送给哪些专家。它会为每个专家计算一个权重分数，然后选择权重最高的前k个专家（通常k=1或2）。

6.3 工作流程

1. 输入：一个数据向量被送入MoE层。
2. 路由：门控网络接收这个输入，并计算每个专家对于该输入的“适用度”分数。
3. 选择专家：门控网络选择分数最高的k个专家。例如，k=2，则选择top-2专家。
4. 专家计算：输入数据被分别发送给这k个被选中的专家。每个专家独立地进行计算，产生自己的输出。
5. 加权组合：将k个专家的输出结果，按照他们从门控网络得到的分数进行加权求和，得到最终的MoE层输出。

虽然模型的总参数量非常大（因为有很多专家），但每个输入实际激活的参数量（称为激活参数）只是被选中的k个专家的参数，加上门控网络。这实现了“用更少的计算，撬动更大的模型能力”。

6.4 MoE的优势

1. 极高的计算效率：这是最突出的优点。模型总参数可以达到千亿甚至万亿规模，但每个token的计算成本只相当于一个百亿规模的稠密模型。这意味着用更少的计算资源，获得了大模型才具备的强大能力。
2. 模型容量巨大：MoE可以轻松地通过增加专家数量来扩展模型的总知识容量，而不会显著增加计算时间。
3. 隐性专业化：在训练过程中，专家会自发地形成专业化分工。有些专家可能擅长处理数学逻辑，有些擅长文学创作，有些擅长代码生成。门控网络学会了如何将问题分配给合适的专家。

6.5 MoE的挑战

MoE并非完美，它也带来了一些新的挑战：

1. 训练不稳定：门控网络和专家是共同训练的。容易出现“赢家通吃”的局面，即少数几个专家被频繁选择，而其他专家得不到训练，成为“僵尸专家”。需要复杂的负载均衡技术来确保所有专家都能被充分训练。
2. 高昂的显存占用：尽管计算量小，但所有专家的参数都需要加载到GPU显存中。因此，MoE模型对显存的需求非常高。这更像是一种“用显存换计算”的权衡。
3. 通信成本：在分布式训练中，不同的专家可能分布在不同的GPU上。需要高效的数据路由和同步机制，这可能会引入通信瓶颈。

7 FP8混合精度

7.1 FP8混合精度的优势

随着模型变得极大（如大型语言模型拥有千亿参数），即使FP16混合精度也面临巨大挑战：

- 内存瓶颈：模型的权重、激活值、优化器状态和梯度需要消耗巨量GPU显存。FP16相比FP32只能将内存占用减半。
- 算力瓶颈：数据传输（例如从显存到计算单元）成为新的瓶颈，更低的精度意味着传输数据量更小。
- 功耗瓶颈：低精度计算能耗更低。

FP8（8位浮点数）应运而生。它将每个数值的存储空间从FP16的16位进一步压缩到8位，带来了直接的好处：

- 内存占用减半：相比FP16，FP8的权重、激活值占用的内存再减少50%。
- 传输带宽翻倍：在相同的内存带宽下，可以传输两倍于FP16的数据。
- 计算速度提升：新一代硬件（如NVIDIA H100 GPU）开始原生支持FP8计算，其计算吞吐量是FP16的两倍。

7.2 使用方式

直接使用FP8训练会非常困难，因为8位的数值表示范围和解精度比FP16更小，更容易出现数值不稳定。因此，FP8训练不是简单地把所有数据变成8位，而是一套更精细的系统工程。

其核心是“混合”和“动态缩放”。

两种FP8格式：E5M2 和 E4M3

为了解决FP8表示范围和解精度之间的矛盾，业界提出了两种主流的FP8格式：

- E5M2：5位指数 + 2位尾数。优势是表示范围大，不易溢出，适合存储梯度等可能数值范围较大的数据。
- E4M3：4位指数 + 3位尾数。优势是精度稍高（因为尾数多了一位），但表示范围较小，适合存储权重和激活值。

在典型的FP8训练流水线中，可能会混合使用这两种格式，例如用E4M3表示前向传播的权重和激活，用E5M2表示反向传播的梯度。

分层动态缩放

这是FP8训练的灵魂。由于FP8的“尺子”刻度非常稀疏，我们必须确保要测量的“物体”（权重、激活、梯度）能够被精确地放在这把尺子上。

- 缩放因子：为一个张量（比如一层网络的输出激活值）计算一个缩放因子。这个因子是一个简单的浮点数，用于将该张量的数据范围“映射”到FP8能够有效表示的范围之内。
- 动态性：缩放因子不是固定的，而是在每个训练步骤中动态计算的。因为激活值和梯度的分布会随着模型权重的更新而不断变化。算法会跟踪张量的历史绝对值最大值（或使用其他统计量）来实时调整缩放因子。
- 粒度：缩放可以以非常细的粒度进行，例如每个张量甚至每个通道都有自己独立的缩放因子。这比FP16时代通常使用单个全局缩放因子要精细得多，能更好地保持数值精度。